SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I Sanjana.G (192511020) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Sanjana.G

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

1. AI-Powered Drone Delivery Using Greedy Best-First Search and A*

i. Greedy Best-First Search:

Greedy Best-First Search selects the path that appears closest to the destination using a heuristic function \(h(n)\). In this scenario, the drone chooses the route with the shortest straight-line distance to the emergency location.

Strengths:

Makes quick decisions using heuristic estimates.

Explores fewer paths in many situations.

Suitable for urgent medicine delivery.

Weaknesses:

May select blocked or dangerous routes.

Ignores the actual cost of travelling.

Does not guarantee the shortest or safest path.

ii. A Search Algorithm:*

A* considers both the actual cost and the estimated remaining cost to reach the destination. It uses the formula:

\[f(n)=g(n)+h(n) \]

Where:

\(g(n)\) = Actual cost from the starting point.

\(h(n)\) = Estimated cost to the destination.

\(f(n)\) = Total estimated path cost.

A* selects the path with the lowest estimated total cost. It can consider distance, weather, terrain, and travel delay when calculating the path cost.

iii. Impact of Heuristic Accuracy:

An accurate heuristic helps both algorithms select better routes. Greedy Best-First Search depends heavily on the heuristic and may make poor decisions if the estimate is misleading. A* combines the heuristic with actual travel costs, making it more reliable. An admissible heuristic helps A* find an optimal path when its other required conditions are satisfied.

iv. Impact of Dynamic Changes:

Sudden roadblocks or weather changes can make the selected route unsuitable. Both algorithms may need to search again. Repeated searches can increase processing time and delay delivery. Updating the map and using incremental replanning can help the drone respond quickly.

v. Recommended Algorithm:

A* is more suitable because it balances actual travel cost and estimated remaining cost. It can find a low-cost route while considering weather and terrain. For real-time safety, it should be combined with frequent updates, obstacle detection, and rapid replanning. This ensures reliable medicine delivery while reducing delay and risk.

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Recommend the best approach and justify your choice based on scalability and adaptability

Smart City Traffic Signal Optimization

Formulation as an Optimization Problem:

The objective is to minimize total waiting time, fuel consumption, and traffic congestion by adjusting signal timings at multiple intersections.

The objective function can be represented as:

\[\min J=w_1W+w_2F+w_3C \]

Where:

\(W\) = Total waiting time.

\(F\) = Fuel consumption.

\(C\) = Traffic congestion.

\(w_1,w_2,w_3\) = Importance weights.

Traditional search methods are inefficient because the search space is extremely large, traffic conditions change continuously, and evaluating every possible signal combination requires significant computation.

i. Hill Climbing Algorithm:

Hill Climbing starts with an initial signal-timing arrangement and evaluates nearby arrangements. It selects a better arrangement at each step until no further improvement is found.

Advantages:

Simple to implement.

Requires less memory.

Quickly improves an existing solution.

Limitations:

May get stuck in a locally optimal solution.

Does not guarantee a global optimum.

May struggle with rapidly changing traffic patterns.

ii. Local Maxima, Plateaus, and Ridges:

Local maxima: A signal arrangement appears best among nearby options but is not the best overall arrangement.

Plateaus: Several arrangements produce similar results, making it difficult to identify a better solution.

Ridges: The best improvement requires coordinated changes to several intersections, but individual changes may not appear beneficial.

iii. Simulated Annealing and Genetic Algorithms:

Simulated Annealing: It sometimes accepts worse solutions to escape local optima. This helps explore different signal arrangements and reduces the chance of getting trapped.

Genetic Algorithm: It maintains a population of possible signal arrangements and improves them using selection, crossover, and mutation. It can explore a large search space and find effective combinations of timings.

iv. Recommended Approach:

A Genetic Algorithm is suitable for optimizing signal timings across hundreds of intersections because it explores many possible solutions simultaneously. It can balance multiple objectives and adapt to changing traffic data by periodically updating its population. Real-time performance can be improved by using parallel processing, limiting computation time, and applying the best solution found so far.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

Justify the most suitable approach considering uncertainty, adaptability, and safety

i. Need for Online Search:

Online search is required because the Mars rover explores unknown terrain without a complete map. It must make decisions while moving and learn about its surroundings through sensors.

Offline search requires a sufficiently known environment before planning, whereas online search allows the rover to discover new paths and obstacles during exploration.

ii. Challenges of Partial Observability and Dynamic Environments:

The rover has limited sensor information.

Some rocks, craters, and hazards may remain hidden.

Terrain conditions may differ from initial estimates.

Communication delays limit immediate human assistance.

Incorrect decisions can damage the rover or waste energy.

iii. Building and Updating the Internal Model:

The rover uses sensors and cameras to observe its surroundings. It stores discovered terrain, obstacles, possible paths, and estimated travel costs in an internal map. After each movement, it updates the map using new observations and revises its route when necessary.

iv. Comparison of Search Strategies:

Uninformed search: Explores without using estimates of the distance to the goal. It may waste time and energy in unknown terrain.

Informed search: Uses heuristic information to guide exploration toward promising locations. However, it depends on the quality of the estimates.

Online search: Combines movement, observation, and planning. It can revise decisions as new information becomes available.

v. Recommended Approach:

An online search agent using heuristic guidance is most suitable. It can use A*-based planning on the currently known map, update its knowledge continuously, and replan when

hazards are discovered. Safety constraints, risk estimation, and emergency stopping should also be included to ensure safe exploration under uncertainty.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others,

Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation
ii. Explain how backtracking search works in this scenario
iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
iv. Analyze real-world challenges and justify the best strategy for scalability

Formulation as a CSP:

A Constraint Satisfaction Problem consists of variables, domains, and constraints. The university timetable can be represented using these three components.

i. Variables, Domains, and Constraints:

Variables: Each course or examination is represented as a variable.

Example: \(E_1,E_2,E_3,\ldots,E_n\).

Domains: Each variable has possible examination time slots and examination halls.

Example: Monday morning, Monday afternoon, Tuesday morning, and available halls.

Constraints: These are rules that every valid timetable must satisfy.

No overlapping exams: Students must not have two examinations at the same time.

Hall capacity: The number of students must not exceed hall capacity.

Hall availability: A hall cannot host multiple exams simultaneously.

Subject ordering: Prerequisite subjects must be scheduled before dependent subjects.

Faculty availability: Examinations must follow faculty availability requirements.

Special accommodations: Students requiring special facilities must be assigned suitable halls and time slots.

The goal is to find an assignment that satisfies all hard constraints while minimizing conflicts and inconvenient schedules.

ii. Backtracking Search:

Backtracking assigns a time slot and hall to one examination at a time. After each assignment, it checks whether the constraints are satisfied. If a conflict occurs, it removes the assignment and tries another option. This process continues until a valid timetable is found or all possible assignments are exhausted.

iii. Constraint Propagation:

Constraint propagation reduces the possible values of unassigned variables after each assignment. For example, if a student has an exam on Monday morning, conflicting examinations are removed from that time slot.

Forward checking detects conflicts early and avoids exploring invalid combinations. This reduces unnecessary backtracking and improves efficiency.

iv. Real-World Challenges and Best Strategy:

Large universities may have thousands of students, many courses, limited halls, and complex scheduling rules. These increase the search space and computational cost.

A combination of backtracking, constraint propagation, the Minimum Remaining Values (MRV) heuristic, and degree heuristic is suitable. MRV selects the examination with the fewest available options, while the degree heuristic prioritizes examinations that constrain the most others. These techniques reduce search effort and improve scalability.

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

1. Explain how the Minimax algorithm models this problem.

The Minimax algorithm helps the AI choose the best move by considering the possible moves of both the AI and the human opponent.

- Maximizer: AI tries to maximize its score and chances of winning.
- Minimizer: Human opponent tries to minimize the AI's score.
- Game tree: Represents all possible moves and future game states.
- Decision-making: The AI selects the move that provides the best possible outcome, assuming the opponent also plays intelligently.

Thus, Minimax helps the AI make strategic decisions by predicting future moves.

2. Analyze the computational challenges of large game trees.

Large game trees create several computational challenges:

- High branching factor: Each game state may have many possible moves.
- Increased depth: Exploring more future moves increases computation time.
- Memory usage: Storing game states requires additional memory.
- Real-time constraints: The AI must decide within a limited time.
- Complex evaluation: Analyzing resources, units, territory, and opponent actions increases processing requirements.

Therefore, exploring the entire game tree is often impractical.

3. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

- Alpha (α): The best value currently guaranteed to the maximizer.
- Beta (β): The best value currently guaranteed to the minimizer.
- Pruning condition: When $\alpha \geq \beta$, the remaining branches can be ignored.
- Reduced computation: Fewer game states need to be evaluated.
- Same result: It produces the same optimal move as Minimax when both use the same search depth and evaluation assumptions.

Example: If the AI has already found a move with a value of 8, and another branch cannot produce a better outcome, that branch can be pruned.

Thus, Alpha-Beta pruning saves time and allows the AI to explore deeper game trees efficiently.

4. Design an evaluation function and justify the factors considered.

An evaluation function estimates the strength of a game state using important factors.

$$\begin{aligned} E(s) ={}& 5M + 4R + 3T\\ &+ 6B + 2D \end{aligned}$$
Where:

- M = Military strength.
- R = Available resources.
- T = Territory controlled.
- B = Base strength and safety.
- D = Defensive advantage.

Justification:

- Military strength: Determines the ability to attack and defend.
- Resources: Support unit production and upgrades.
- Territory: Provides strategic control and expansion opportunities.
- Base strength: Protects important structures and reduces the risk of defeat.
- Defensive advantage: Helps the AI survive enemy attacks.

The weights can be adjusted through testing. The function should also account for the opponent's strength, and higher scores should represent states more favorable to the AI.

5. Recommend a strategy for balancing optimality and real-time performance.

The AI should combine Minimax, Alpha-Beta pruning, and heuristic evaluation to make fast and effective decisions.

- Alpha-Beta pruning: Removes unnecessary branches.
- Iterative deepening: Searches progressively deeper while tracking the best move found so far.
- Move ordering: Examines promising moves first to improve pruning.
- Depth limitation: Restricts search depth according to the available time.
- Heuristic evaluation: Estimates states without exploring every future move.
- Time management: Stops searching before the deadline and returns the best move found.

Conclusion: Combining Minimax with Alpha-Beta pruning, iterative deepening, and a suitable evaluation function provides a strong balance between decision quality and real-time performance.

Think

To pick up a draggable item, press the space bar. While dragging, use the arrow keys to move the item. Press space again to drop the item in its new position, or press escape to cancel.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand